CDC SystemVerilog Assertion

Interview Questions & Correct Answers

Q1 CDC Transfer Verification Through 2-Flop Synchronizer

Problem

Verify that when A_in changes in the clk_A domain, its value is correctly transferred through two synchronizer flops and appears at B_out in the clk_B domain.

Correct Answer

```
property p_cdc_transfer;
  local logic v_temp;
  @(posedge clk_A)
    ($changed(A_in), v_temp = A_in)
  |=>
    @(posedge clk_B)
      ##1 (B1_out === v_temp)   // first sync flop
      ##1 (B_out  === v_temp);  // second sync flop
endproperty

assert property(p_cdc_transfer);
```

Key Points

- Use local to declare per-thread captured variables.
- |=> (not |->) correctly hands off evaluation after capturing the value.
- Switch to @(posedge clk_B) in the consequent to evaluate in the RX domain.
- Use fixed ##1 delays matching each synchronizer stage; avoid ranges like ##[2:3].
- Use === (case equality) to correctly handle X values in formal verification.

Q2 CDC Pulse Width — Fast-to-Slow Domain

Problem

Ensure a pulse from a fast source clock domain is wide enough to be captured by a slower destination clock. Rule: pulse width >= 1.5x the destination clock period.

Correct Answer

```
property p_pulse_width;
  @(posedge clk_src)
    $rose(pulse) |=> pulse[*2];
endproperty

assert property(p_pulse_width);

// For clk_src : clk_dst = N : 1, use pulse[*N]
// Example for 4:1 ratio:
// $rose(pulse) |=> pulse[*4];
```

Key Points

- Use |=> so the repetition starts the cycle AFTER the rise, measuring 2 full high cycles.
- pulse[*2] means pulse stays HIGH for 2 consecutive clk_src cycles after the rise.
- Adjust [*N] to match the actual clock ratio; N must satisfy N >= 1.5 * (Tsrc/Tdst).

Q3 Signal Active for Exactly 2–3 Cycles

Problem

Check that a signal stays HIGH for exactly 2 to 3 clock cycles after it rises, then goes LOW.

Correct Answer

```
property p_exact_pulse;
  @(posedge clk)
    $rose(sig) |->
      sig[*2:3] ##1 !sig;  // HIGH for 2-3 cycles, then LOW
endproperty

assert property(p_exact_pulse);
```

Key Points

- sig[*2:3] checks 2-3 consecutive HIGH cycles including the $rose cycle.
- ##1 !sig enforces the falling edge — without it, a permanently HIGH signal vacuously passes.
- Use |-> so the $rose cycle is included in the count; use |=> if you want 2-3 extra cycles after the rise.

Q4 Every Request Eventually Acknowledged (Liveness)

Problem

Ensure every request asserted in clk_A is eventually acknowledged in clk_B. No request may be silently dropped.

Correct Answer

```
property p_req_ack;
  @(posedge clk_A)
    $rose(req) |-> ##[1:MAX_LATENCY] ack_sync;
endproperty

assert property(p_req_ack);
// Pair with p_req_hold (Q5) for complete handshake coverage.
```

Key Points

- Trigger on $rose(req) to avoid re-firing every cycle req is held high.
- Use a bounded timeout ##[1:MAX_LATENCY] to catch livelock in simulation.
- ack_sync must be the synchronized version of ack in the clk_A domain.
- Pair with the req-hold assertion (Q5) for complete handshake verification.

Q5 Request Held Until Acknowledgement (Safety)

Problem

Ensure the request signal remains continuously asserted until the synchronized acknowledgement arrives. Prevents early de-assertion that violates the handshake protocol.

Correct Answer

```
property p_req_hold;
  @(posedge clk_A)
    $rose(req) |-> req until_with ack_sync;
endproperty

assert property(p_req_hold);
```

Key Points

- until_with is the strong operator: req must stay HIGH AND ack_sync must eventually arrive.
- until (weak) passes vacuously if ack_sync never comes — incorrect for protocol checking.
- Trigger on $rose(req) to start the check only when req first asserts.

Q6 Two-Flop Synchronizer Data Integrity

Problem

Verify that a CDC signal correctly passes through a two-flop synchronizer: capture the TX-domain value and confirm it arrives intact at the final synchronized output.

Correct Answer

```
property p_two_flop_sync;
  local logic captured;
  @(posedge clk_src) $changed(tx_data)
  |=> (1'b1, captured = tx_data)
    ##1 @(posedge clk_dst)
      ##1 (sig_meta === captured)  // first flop
      ##1 (sig_sync === captured); // second flop
endproperty

assert property(p_two_flop_sync);
```

Key Points

- Capture tx_data value in the TX domain using a local variable.
- 1'b1 is the required dummy expression to attach the local variable assignment.
- Switch to @(posedge clk_dst) in the consequent to evaluate in the correct domain.
- Check both synchronizer stages (sig_meta then sig_sync) with fixed ##1 delays.
- Structural topology verification (does 2-flop exist?) belongs in a static CDC tool.

Q7 CDC Signal Stability — No Back-to-Back Changes

Problem

Ensure the CDC signal does not change on consecutive clock cycles. Back-to-back transitions reduce the metastability resolution window and can cause missed captures.

Correct Answer

```
property p_cdc_stability;
  @(posedge clk_src)
    $changed(sig) |=> $stable(sig);
endproperty

assert property(p_cdc_stability);

// For minimum pulse width enforcement:
property p_min_pulse_width;
  @(posedge clk_src)
    $rose(sig) |=> sig[*MIN_HOLD_CYCLES];
endproperty
assert property(p_min_pulse_width);
```

Key Points

- This is a stability check, not a sub-cycle glitch detector.
- $stable(sig) in the consequent is clearer than !$changed(sig).
- Sub-cycle glitches require gate-level simulation with SDF timing — not clocked SVA.
- Combine with the pulse-width assertion to fully constrain CDC signal transitions.

Q8 Gray Code FIFO Pointer — One-Bit Change Per Cycle

Problem

Verify the FIFO pointer is Gray-coded: it must change by at most one bit per clock cycle, ensuring safe 2-flop synchronization across clock domains.

Correct Answer

```
property p_gray_code;
  @(posedge clk) disable iff (!rst_n)
    $onehot0(ptr ^ $past(ptr));
endproperty

assert property(p_gray_code);
```

Key Points

- ptr ^ $past(ptr) produces a bitmask of all bits that changed since last cycle.
- $onehot0 passes if exactly 0 or 1 bits are set — the Gray code invariant.
- disable iff (!rst_n) is essential: during reset the pointer may jump non-Gray back to 0.

Q9 Async FIFO — Full and Empty Are Mutually Exclusive

Problem

Ensure the asynchronous FIFO is never simultaneously full and empty. Since full and empty are computed in separate clock domains, each must be checked in its own domain.

Correct Answer

```
// full is valid in wclk domain
property p_not_full_and_empty_w;
  @(posedge wclk) disable iff (!wclk_rst_n)
    full |-> !empty_sync;
endproperty
assert property(p_not_full_and_empty_w);

// empty is valid in rclk domain
property p_not_full_and_empty_r;
  @(posedge rclk) disable iff (!rclk_rst_n)
    empty |-> !full_sync;
endproperty
assert property(p_not_full_and_empty_r);
```

Key Points

- An async FIFO has no single shared clock — never mix full and empty on one posedge clk.
- full is generated in the wclk domain; empty is generated in the rclk domain.
- Use the synchronized counterpart (empty_sync, full_sync) when crossing domains.
- Add disable iff for each domain's reset signal independently.

Q10 Async FIFO — Full Flag Resolves Within CDC Sync Latency

Problem

After full asserts in the wclk domain, the synchronized empty_sync must deassert within the synchronizer latency window. Accounts for CDC propagation delay.

Correct Answer

```
// Allow CDC sync delay before checking consistency
property p_full_resolves;
  @(posedge wclk) disable iff (!wclk_rst_n)
    $rose(full) |-> ##[1:SYNC_LATENCY+1] !empty_sync;
endproperty
assert property(p_full_resolves);

// SYNC_LATENCY = number of sync flop stages (typically 2)
```

Key Points

- Trigger on $rose(full) to check only when full first asserts.
- ##[1:SYNC_LATENCY+1] allows the synchronized pointer time to propagate.
- Check in wclk domain using empty_sync (the rclk empty flag synced to wclk).
- Typical SYNC_LATENCY is 2 for a standard 2-flop synchronizer.

Q11 Multi-Bit Data Safe Sampling via Enable Synchronization

Problem

Ensure multi-bit data_bus is stable when the synchronized enable is asserted in the destination domain, preventing capture of a transitioning (torn) bus.

Correct Answer

```
// When enable_sync is HIGH, data_bus must be stable for safe capture
property p_multibit_safe;
  @(posedge clk_dst) disable iff (!rst_n)
    enable_sync |-> $stable(data_bus);
endproperty
assert property(p_multibit_safe);

// No update to captured register when enable is LOW
property p_capture_on_enable;
  @(posedge clk_dst) disable iff (!rst_n)
    !enable_sync |-> $stable(captured_data);
endproperty
assert property(p_capture_on_enable);
```

Key Points

- enable_sync |-> $stable(data_bus): bus must be stable when enable is asserted.
- $stable(sig) is true if sig has the same value as the previous clock edge.
- The enable handshake ensures the TX side holds data_bus stable during capture.
- The second property ensures captured_data is only updated when enable_sync is HIGH.

Q12 Async Control Signal — Synchronized Before Use

Problem

Verify that an asynchronous control signal is passed through a 2-flop synchronizer before use in the destination domain. The TX-domain source value must match the final synchronized output.

Correct Answer

```
property p_sync_correctness;
  local logic cap;
  @(posedge clk_src) $changed(ctrl_src)
  |=> (1'b1, cap = ctrl_src)
    ##1 @(posedge clk_dst)
      ##2 (ctrl_sync === cap);  // 2-flop sync delay
endproperty

assert property(p_sync_correctness);

// Structural check (2-flop topology exists) -> use static CDC tool,
// e.g. Synopsys SpyGlass CDC or Mentor Questa CDC.
```

Key Points

- Never sample ctrl_async in a clk_dst assertion — that itself is a CDC violation.
- Capture the source value in clk_src domain; verify it after exactly ##2 clk_dst cycles.
- ##2 corresponds to a 2-flop synchronizer; use ##3 for a 3-flop synchronizer.
- Structural verification (does the synchronizer topology exist?) must be done by a static CDC tool.